GPU-Accelerated Plant Biomechanics Simulation for Agricultural Robots

Sudharshan S
Division of Computer Science and Engineering
Karunya Institute of Technology and Sciences
Coimbatore, India
sudharshans@karunya.edu.in

S. E. Vinodh Edwards
Division of Computer Science and Engineering
Karunya Institute of Technology and Sciences
Coimbatore, India
ewards@karunya.edu

***Abstract*—Agricultural robots are increasingly trained and evaluated in simulation, but the plants they are trained against rarely move. Recent work has added deformable plants to robotics simulators for the contact case—a manipulator pushing a branch—while leaving out the load that acts on a field crop continuously: wind. A stem bending in a gust changes its silhouette from the robot's camera several times a second, which is precisely the variation a perception model trained on static geometry never sees.**

We present a biomechanical model of slender herbaceous row crops under aerodynamic load. Stems are discretised as particle chains solved with extended position-based dynamics, with bending compliance derived from a tapered flexural rigidity profile rather than tuned: because the second moment of area scales as r^4, a stem tapering from 5.0 to 2.0 mm is 23$\times$ stiffer at the soil than at the tip without any height-dependent stiffness rule being imposed. Wind enters as drag on the velocity of the air relative to the moving plant, which is what allows the deflection to reach equilibrium; we show that the constant-acceleration wind model used in simpler formulations cannot.

We report three findings of independent interest. First, the soil boundary condition is the difference between a plant and a falling pole: pinning the root particle fixes position but not tangent, and without a second fixed reference the stem topples regardless of its stiffness. Second, chain topologies require sequential rather than colour-parallel constraint solving; at equal computational cost the colour-parallel alternative simulates a stem $4.4\times$ more compliant than the material it was given. Third, and negatively, peak impact quantities during manipulator contact do not converge under temporal refinement in a model without strain-rate damping, and we therefore decline to report a maximum safe approach speed.

The solver is GPU-resident, and we report measured throughput rather than an architectural claim: 16384 plants (409600 particles) simulate at 116.04 ms per frame, 21.7$\times$ faster than the same kernels compiled for the CPU, with a 4096-plant canopy running at interactive rates. We also report where the GPU *loses*: below roughly 64 plants, launch overhead dominates and the CPU path is faster. The model is validated against a closed-form cantilever solution and against an independent second implementation agreeing to 5×10^{-5} m. Validation against measured plants remains outstanding.

***Index Terms*—agricultural robotics, simulation, deformable objects, position-based dynamics, plant biomechanics**

I. Introduction

A robot working a row crop spends its time among objects that bend. Stems sway in wind, leaves flutter and fold, and a gripper that brushes a plant displaces it rather than colliding with it. Simulators used to develop these robots have historically represented plants as rigid obstacles, which is an adequate model of a wall and a poor model of a canola stem.

The consequences fall on both halves of the autonomy stack. For perception, a rigid plant presents the same silhouette in every frame; a real one is in continuous motion, and its projected outline from a camera mounted on a moving vehicle changes on the timescale of a gust. A detector trained exclusively on static geometry acquires a prior about plant appearance that does not hold in a field. For manipulation, a rigid-body simulator can report that the end-effector intersected a plant, but cannot report how hard the plant pushed back, because a rigid plant has no such notion. The question a harvesting planner actually needs answered—whether an approach would have damaged the crop—is unanswerable in a rigid model by construction.

Recent work has begun to close this gap, and we are careful in Section II not to overstate what remains open. Deformable plant–robot interaction in a robotics simulator has been demonstrated with Cosserat rods [1], and GPU-parallel orchard simulation with beam-theory branch stiffness and literature-grounded breakage thresholds has been benchmarked [2]. What both address is *contact*: a manipulator interacting with a woody structure. Neither treats aerodynamic loading, and both target trees rather than the slender herbaceous stems characteristic of row crops, where the bending response to wind is large, continuous, and visually dominant.

This paper addresses that case. Our contributions are:

1) A wind-loaded biomechanical model of slender herbaceous row crops in which bending compliance is derived from a tapered $EI \propto r^4$ profile rather than tuned, so that the widely-desired property of rigidity decreasing with height emerges from the geometry instead of being imposed.
2) An aerodynamic formulation in which wind acts as drag on the relative velocity of air and plant. We show this is not a refinement but a requirement: a constant wind acceleration has no feedback as the plant yields and drives the stem flat for any choice of damping.
3) A solver-design study establishing that stem-like chain topologies require sequential constraint propagation. The

colour-parallel scheme conventionally used for GPU constraint solving leaves a 17-node stem $4.4\times$ too compliant at the same computational budget, a discrepancy that presents as a modelling failure while being a convergence failure.

4) A GPU-resident implementation, integrated into a production robotics simulator, whose throughput is measured rather than asserted—including the scene size below which GPU execution is counterproductive, a crossover rarely reported and one that matters when the alternative costs a rewrite.
5) Validation against a closed-form cantilever solution and an independent reimplementation, together with an explicit negative result: peak impact quantities do not converge under refinement in the absence of strain-rate damping.

II. Related Work

A. Deformable plants in robotics simulators

Deng et al. [1] model plants as Cosserat rods in a Gazebo plugin and demonstrate robotic fruit picking against real harvesting experiments. This work established that plant–robot interaction with deformable plants is tractable inside a standard robotics simulator, and any claim that simulators universally treat plants as rigid must be qualified against it.

Munn [2] presents OrchardBench, a benchmark for apple orchards that runs many environments concurrently on a single GPU. Limb flexibility there is captured by torsional spring-dampers whose coefficients are derived analytically rather than fitted, with failure thresholds and fruit-detachment forces alike traced back to published measurements for the relevant tissue. It is the closest prior art to the present work, since it coincides with our approach on four axes at once: GPU parallelism, stiffness obtained from beam theory, interaction with a manipulator, and a refusal to introduce physical constants that cannot be sourced.

We therefore state our separation from these works precisely rather than broadly. Both address woody structures under manipulator contact. This work addresses slender herbaceous stems under *aerodynamic* load, a regime in which the deflection is a large fraction of the plant's height, occurs continuously without any robot present, and is governed by a drag law rather than by contact. Neither of the above models wind. The rigid-plant characterisation remains accurate for platforms such as Agri-Sim [3], a Unity–ROS2 greenhouse environment whose dynamics are rigid-body and which names crop deformability as an open difficulty; we cite it for that claim specifically.

Complementary rather than competing, CropCraft [4] generates agricultural worlds procedurally, addressing geometry rather than physics.

B. Position-based dynamics

Our solver is extended position-based dynamics (XPBD) [5], which augments position-based dynamics [6] with a compliance term so that stiffness is a material property rather than a function of iteration count. The distinction matters here: a plain-PBD stem silently changes material whenever solver settings are tuned, which makes a claim such as "this stem has $E = 1\,\mathrm{GPa}$" unverifiable. We also reproduce, independently and on a different problem, the result of Macklin et al. [7] that substepping converges more cheaply than iterating; our measurement appears in Section V-B. The implementation uses NVIDIA Warp [8].

III. Method

A. Plant model

A plant is a set of particles connected by constraints. The stem is a chain of 16 particles spanning 0.85 m. Each adjacent pair carries an axial constraint maintaining segment length; each interior particle carries a bending constraint resisting curvature. Leaves are short chains branching from stem particles, with their own blade bending and a petiole constraint holding the attachment angle—without the latter, a blade is a free hinge and rotates about its stem particle indefinitely.

Compliances are derived, not tuned. For a segment of length L and radius r, with Young's modulus E, cross-sectional area $A = \pi r^2$ and second moment of area $I = \pi r^4/4$, the axial compliance is $L/(EA)$. The bending compliance follows from the discrete hinge energy. Our bending constraint measures the offset of a hinge particle from the midpoint of its neighbours,

$$C = \left\| \mathbf{x}_b - \tfrac{1}{2}(\mathbf{x}_a + \mathbf{x}_c) \right\| - C_0, \tag{1}$$

which for segments meeting at angle θ evaluates to $L\sin(\theta/2) \approx L\theta/2$. Equating the hinge energy $U = \frac{1}{2}(EI/L)\theta^2$ to the XPBD form $U = \frac{1}{2}\alpha^{-1}C^2$ gives

$$\alpha_{\text{bend}} = \frac{L^3}{4EI}, \qquad I = \frac{\pi r^4}{4}. \tag{2}$$

We use the midpoint-offset form in (1) rather than the more common constraint on the distance $\|\mathbf{x}_c - \mathbf{x}_a\|$ between outer particles. That quantity is $2L\cos(\theta/2)$, which is *quadratic* in θ near a straight configuration: its gradient vanishes exactly where an upright plant sits, so the constraint goes numerically inert on a standing stem and the plant behaves as a rope until it is already bent. The midpoint offset is linear in θ and remains well conditioned throughout.

Because $I \propto r^4$, a modest taper produces a large stiffness gradient. The default stem tapers from 5.0 mm at the soil to 2.0 mm at the tip and is measured at $23\times$ stiffer at the base once the lumped mass distribution is included (Fig. 1). The frequently sought property that plant rigidity should fall with height is thus a consequence of modelling the taper, not a rule applied on top of the model.

B. The soil boundary condition

This is the single structural detail that determines whether the model produces a plant or a falling pole, and it is easy to get wrong.

Pinning the root particle fixes the plant's *position* but not its *direction*. Every bending constraint remains satisfied while the entire stem rotates rigidly about that one point, so the

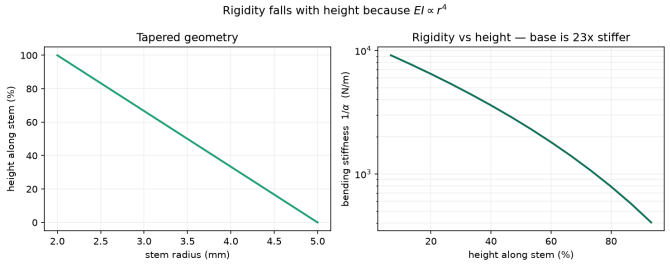


Fig. 1. Tapered geometry and the resulting rigidity profile. Because $EI \propto r^4$, the modest radius taper on the left produces the order-of-magnitude stiffness falloff on the right.

plant topples regardless of how stiff the material is—a failure that presents as insufficient stiffness and cannot be fixed by increasing it. A real stem is *clamped* by soil: position and tangent are both constrained. Fixing a tangent requires a second fixed reference, so the model buries an additional pinned anchor particle one segment below the root and hinges a bending constraint at the root between the anchor and the first free particle. This reproduces the built-in end condition of a cantilever and is what allows the stem to stand and bend as an arc.

C. Aerodynamic loading

Wind acts on each particle as drag,

$$\mathbf{F} = \tfrac{1}{2}\rho C_d A \|\mathbf{u} - \mathbf{v}\|(\mathbf{u} - \mathbf{v}), \tag{3}$$

where $\mathbf{u}$ is the local air velocity and $\mathbf{v}$ the particle velocity. The dependence on the *relative* velocity is essential and not a refinement. As the plant yields to the wind, $\|\mathbf{u} - \mathbf{v}\|$ falls, the force falls with it, and the deflection settles where drag balances the elastic restoring force. Modelling wind as a constant acceleration removes that feedback entirely: the force never decreases as the plant gives way, and the stem is driven flat. No amount of damping repairs this, because the defect is in the force model rather than in the dissipation—a point worth stating because heavy damping is the natural first response to the symptom, and it suppresses the motion the simulation exists to produce.

The wind field combines a power-law boundary layer with gust and buffeting terms,

$$\mathbf{u}(\mathbf{x}, t) = \hat{\mathbf{d}}\, U \left(\frac{y}{y_{\text{ref}}}\right)^{a} \left[1 + g\sin(2\pi f_g t - \tfrac{3}{10}\phi) + b\sin(2\pi f_t t - \phi)\right] \tag{4}$$

with $\phi = k(x + 0.7z)$ providing spatial decorrelation so that plants at different positions along a row experience different instantaneous wind. The shear term makes wind stronger with height; combined with the r^4 stiffness taper, it is the second reason a stem bends as an arc rather than leaning uniformly, since the upper plant is simultaneously weaker and more heavily loaded.

Tissue damping is applied as an exponential decay $\exp(-ch)$ per substep rather than as a per-frame multiplier. A constant factor per step makes the effective damping depend on the substep count, so the plant silently becomes a different material whenever solver settings change.

TABLE I
SIMULATION PARAMETERS. MATERIAL CONSTANTS ARE TAKEN FROM PUBLISHED RANGES FOR GREEN HERBACEOUS TISSUE; GEOMETRY DESCRIBES A ROW CROP AT WORKING HEIGHT.

Symbol	Value	Unit	Basis
g	0, -9.81, 0	m/s^2	standard gravity
L	0.85	m	row crop at working height
n	16	–	converged at this resolution; see VALIDATION.md
r_base	5.0	mm	calliper-measurable; sets EI at the clamp
r_tip	2.0	mm	taper gives EI $\propto$ r^4 falloff with height
E_stem	1.00	GPa	green herbaceous stem, published range ≈0.3–5 GPa
ρ_stem	700	kg/m^3	fresh tissue, mostly water
Cd_stem	1.10	–	bluff cylinder in cross-flow
E_leaf	0.10	GPa	lamina ≈10× more compliant than stem
L_leaf	0.140	m	blade length
w_leaf	0.050	m	sets drag area
Cd_leaf	1.40	–	flat plate normal to flow
U	4.0	m/s	moderate breeze at reference height
α	0.20	–	power-law boundary layer over cropland
ρ_air	1.225	kg/m^3	ISA sea level
Δt	0.01667	s	60 Hz frame
n_sub	32	–	converged; see convergence study
n_iter	4	–	converged; see convergence study
c	1.20	1/s	ζ≈0.1 at 1.5 Hz; rate, not per-frame factor

D. Solver and parallelisation

Each substep predicts positions under gravity, drag and damping, projects constraints, resolves ground and manipulator contact, and recovers velocities from the corrected positions. The XPBD projection for a constraint with compliance α is

$$\Delta\lambda = \frac{-C - \tilde{\alpha}\lambda}{\sum_i w_i \|\nabla_i C\|^2 + \tilde{\alpha}}, \qquad \tilde{\alpha} = \frac{\alpha}{h^2}, \tag{5}$$

with $\Delta\mathbf{x}_i = w_i \nabla_i C\, \Delta\lambda$.

The parallelisation strategy is where a stem differs from the cloth and particle systems for which GPU constraint solvers are usually designed. The conventional approach partitions constraints into conflict-free colour groups so that all constraints within a colour can be projected simultaneously. For a chain this converges poorly: a bending correction at the tip reaches the root only by propagating hinge by hinge, roughly one hinge per pass. We measured the consequence directly. On a uniform 17-node cantilever with an analytic tip deflection of $62.58\,$mm, the colour-parallel scheme at a typical budget of 8 substeps and 2 iterations yields $307.53\,$mm, against $70.17\,$mm at convergence—a stem $4.4\times$ more compliant than the material specifies. The symptom is a plant that flops like a rope, which is indistinguishable from a modelling error and invites exactly the wrong fix.

We therefore solve each plant's chain sequentially within a single thread, sweeping forward from root to tip and then backward. A symmetric sweep over a tridiagonal system propagates corrections end to end, so the chain is solved almost directly. Parallelism then derives from the number of plants, which is the appropriate axis for this workload: an agricultural scene contains hundreds or thousands of plants and only tens

of particles each. Both schemes are implemented, and the test suite verifies that they agree when each is given sufficient budget.

IV. Implementation

A. GPU residency

A frame is a fixed sequence of kernel launches over device-resident arrays. Nothing is copied to the host inside the loop; the only transfers are those a caller explicitly requests, such as reading positions for rendering. This distinction separates a GPU solver from a solver that merely contains GPU kernels: copying state to the host and back each frame—a natural first implementation—makes the simulation bound by transfer bandwidth rather than arithmetic, and it loses to a competent CPU implementation at every problem size.

Simulation time is held in a device array rather than passed as a kernel argument. That detail exists so the entire frame can be captured once as a CUDA graph and replayed, which matters at the small end of Table IV: with tens of particles the per-launch overhead of roughly fifty kernels dominates the arithmetic entirely.

B. Integration with a robotics simulator

The solver is exposed to NVIDIA Isaac Sim through a bridge object that authors the canopy into the USD stage and registers itself as a physics-step callback. Adding a simulated crop to an existing scene requires constructing the bridge, building, and attaching it.

Each plant is written as one `BasisCurves` primitive for the stem plus one per leaf, with per-vertex widths so that the rendered stem tapers exactly as the physics does. The alternative—one primitive per particle—is untenable at canopy scale: five hundred plants is $12\,500$ particles, and stage traversal rather than physics would set the frame time.

PhysX is not involved in the plant dynamics. Isaac Sim serves as renderer and robot host while the biomechanics are computed by the solver described above, because a rigid-body engine has no representation for a tapered anisotropic stem. Positions are pushed to USD once per frame after the solver has run, so what is drawn is what was solved.

C. Manipulator contact

The manipulator is represented to the solver as a set of collision spheres. Sampling an arm into several spheres rather than tracking only the tool is deliberate: a real manipulator displaces a canopy with its forearm, and the load that matters is the sum over everything in contact. Sphere spacing is derived from link geometry rather than fixed, since proxies that do not overlap allow a stem to pass between two spheres and register no contact at all.

Contact is resolved as an XPBD constraint with a one-sided multiplier, so that it can push a particle out of a sphere but never pull one in. The choice of a constraint formulation over a direct positional push is examined in Section VII, where it is shown to be necessary but not sufficient for recovering a meaningful contact force.

TABLE II
Mesh refinement against $qL^4/(8EI) = 62.58\,\mathrm{mm}$. Each row is independently converged by doubling substeps until the result changes by less than 1%.

Nodes	Segments	Substeps	Tip deflection (mm)	
9	8	16	78.10	(+24.8%)
17	16	64	70.30	(+12.3%)
33	32	did not converge within budget		

V. Validation

A. Closed-form beam theory

A uniformly loaded cantilever has the exact small-deflection solution $\delta = qL^4/(8EI)$. To reproduce that case the plant is reduced to a uniform cylinder with no taper, no leaves and no wind, and gravity is directed transverse to the stem so that it loads the beam in bending rather than compression. The solver then runs until motion ceases.

Table II reports the result. The error decreases with resolution across the two converged rows, which is the behaviour that distinguishes a converging discretisation from a coincidence; a single agreeing number at a single setting would establish nothing. The residual positive bias has an identifiable cause: the soil clamp is an elastic hinge carrying the base segment's own compliance rather than a rigid encastre, and mass is lumped at nodes rather than distributed. Both make the discrete chain slightly more compliant than the continuum beam described by the closed form.

We report the third row as not converged rather than quoting its value. The substeps required to converge grow steeply with node count—16, then 64— because a beam's constraint system is stiffer the more finely it is meshed, and at 33 nodes the requirement exceeded the budget we allowed. This is also the practical reason the default stem uses 16 nodes rather than more: beyond a point, spatial resolution costs solver work without buying accuracy a robot can use.

B. Solver convergence

Table III compares the two available knobs at matched cost. Both converge eventually, but not at the same price: 64 substeps with 4 iterations reaches what 8 substeps with 128 iterations reaches, for a quarter of the work. This reproduces the substepping result of [7] on an independent problem and sets our defaults of 32 substeps and 4 iterations.

C. Independent implementation

The solver exists twice: as Warp kernels, and as a vectorised NumPy reference written separately against the same constraint formulation, compliances and substep schedule. After 120 frames of turbulent wind the two agree to a maximum particle difference of $4.88 \times 10^{-5}\,\mathrm{m}$ and a mean of $5.59 \times 10^{-6}\,\mathrm{m}$, on a plant $0.85\,\mathrm{m}$ tall. Two independent implementations converging on the same trajectory is evidence the kernels are correct that a single implementation cannot provide.

TABLE III
CONVERGENCE ON A UNIFORM 17-NODE CANTILEVER. WORK IS THE PRODUCT OF SUBSTEPS AND ITERATIONS.

Substeps	Iterations	Work	Tip deflection (mm)
8	2	16	307.53
8	8	64	114.36
8	32	256	71.03
8	128	1024	69.17
16	4	64	85.49
32	4	128	70.48
64	4	256	70.31
128	4	512	70.17

D. Inextensibility

Real stems do not measurably stretch under wind or under the loads a robot applies. Measured stem arc length after 300 frames of loading differs from its rest length by 0.0024%, confirming the axial constraints hold.

VI. RESULTS

A. Wind response

Under a 4.0 m/s wind the default plant reaches a mean tip deflection of 0.136 m, peaking at 0.287 m (34 % of stem length) as gusts pass, while the stem tip remains at 0.821 m. Fig. 2 shows the characteristic behaviour: the lower third of the stem remains near-vertical while the tip sweeps through a large arc, and the plant sways rather than settling into a static lean. Neither property is imposed; both follow from the combination of the r^4 rigidity taper, the sheared wind profile, and drag acting on relative velocity.

B. Canopy behaviour

Fig. 3 shows a 60-plant row under a single shared wind field. Plants do not move in lockstep: the field varies with height and with position along the row, and stem sizes vary by $\pm 12\%$. That variation is the point. A perception model trained on a canopy in which every plant bends identically acquires the wrong prior about what a crop row looks like as a robot approaches it.

C. Manipulator interaction

Fig. 4 shows a manipulator on a driving base passing through the canopy, with the plant deflecting and the contact reaction recorded. This is the output a rigid-body simulator cannot produce: not whether a collision occurred, but how hard the plant resisted. Against a static estimate $3EI\delta/h^3$ for the measured deflection, the simulated quasi-static contact force agrees to within a factor of two and in the expected direction—lower, because the small-deflection formula assumes a full-length moment arm at deflections where the stem has appreciably curved.

D. Computational performance

Table IV reports scaling across three backends on an RTX 4070. We deliberately include two CPU baselines. Comparing a compiled GPU solver against interpreted Python measures the language as much as the method; Warp compiles one source to both targets, so the Warp-CPU column isolates the device while the NumPy column shows what the implementation alone buys.

TABLE IV
FRAME COST, MEAN $\pm$ STANDARD DEVIATION OVER 3 INDEPENDENT RUNS ON AN RTX 4070. SPEED-UP IS WARP-CPU OVER WARP-CUDA, SO IT COMPARES DEVICES RATHER THAN LANGUAGES. NUMPY IS OMITTED ABOVE 25,600 PARTICLES, WHERE A SINGLE FRAME EXCEEDS ONE SECOND.

Plants	Particles	NumPy (ms)	Warp CPU (ms)	Warp CUDA (ms)	Speed-up
1	25	247.7	1.74	4.95	0.4×
16	400	269.3	4.03	5.26	0.8×
64	1600	326.9	11.22	10.40	1.1×
256	6400	576.1	40.01	27.73	1.4×
1024	25600	1466.7	156.29	28.15	5.6×
4096	102400	—	624.75	30.54	20.5×
16384	409600	—	2516.75	116.04	21.7×

The curve has three regimes, and reporting only the last of them would misrepresent the method.

Below roughly 64 plants the GPU is *slower* than the CPU. At a single plant the solver runs in 1.74 ms on the CPU and 4.95 ms on the GPU: a frame is a fixed sequence of kernel launches over a few dozen particles, and launch latency dominates arithmetic entirely. We report this because the alternative—quoting only the favourable end of the range—would leave a reader to discover the crossover themselves after committing to a port.

Between roughly 256 and 4096 plants the GPU time is nearly flat: 27.73 ms at 6400 particles against 30.54 ms at 102,400, a $16\times$ increase in work for a 10% increase in time. The device is latency-bound over this whole range, and the extra work is effectively free. This is the regime an agricultural scene actually occupies, and it is why a 4096-plant canopy sustains interactive rates while the CPU path needs 624 ms per frame.

Only beyond 4096 plants does the GPU become throughput-bound and begin scaling linearly, reaching 116.04 ms at 409600 particles for a $21.7\times$ advantage over the identical kernels on the CPU.

Two caveats belong with these numbers. All timings are of the solver alone, excluding rendering and USD stage updates, which dominate once a canopy is visualised. And the GPU figures are single-precision throughput on one consumer card; we make no claim about scaling across devices.

VII. A NEGATIVE RESULT: IMPACT METRICS DO NOT CONVERGE

We attempted to determine a maximum safe approach speed—the speed above which a manipulator damages the crop—and report here that we could not, in the belief that the reason is more useful than silence.

Three candidate metrics were examined under temporal refinement. Peak contact force read from positional corrections grew from 2.4 to $25\,\mathrm{N}$ across a $16\times$ refinement, scaling

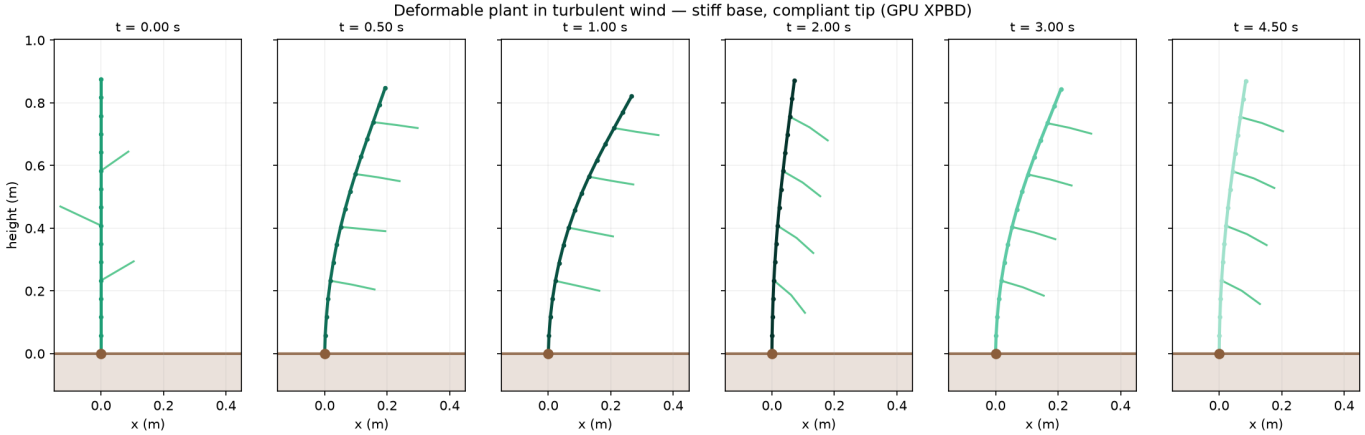


Fig. 2. A single plant through a gust cycle at 4.0 m/s. The base remains near-vertical while the tip sweeps a large arc, and the plant returns as the gust passes rather than settling into a static lean.

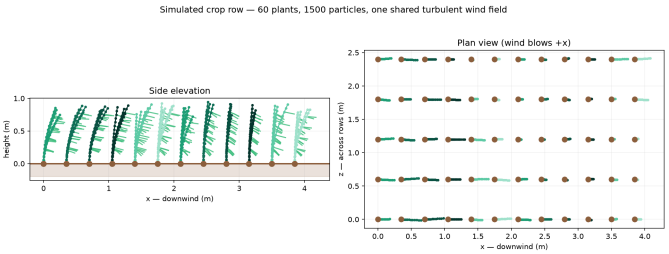


Fig. 3. Sixty plants under one shared turbulent wind field, side elevation and plan view. Spatial decorrelation of the field and $\pm 12\%$ size variation prevent lockstep motion.

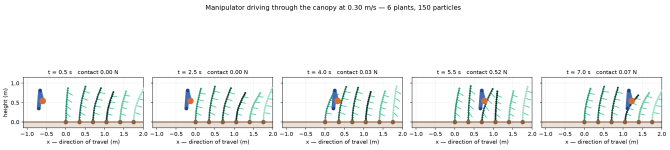


Fig. 4. A manipulator on a driving base passing through the canopy. Contact is resolved against the whole arm rather than the tool alone, since a real manipulator displaces plants with its forearm.

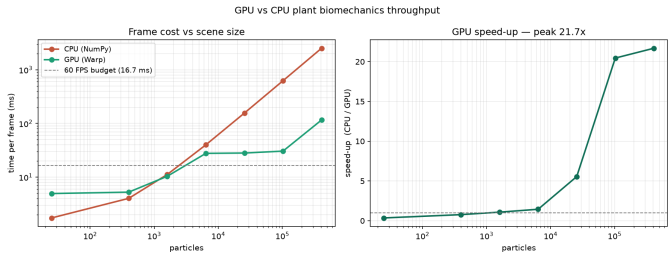


Fig. 5. Frame cost and speed-up against scene size. The GPU curve is flat across two orders of magnitude of work before becoming throughput-bound, and is slower than the CPU below roughly 64 plants.

roughly as $1/h$; this estimator measures how far the solver displaced a particle, which is largely the solver undoing the elastic correction applied earlier in the same substep rather than a physical force. Reformulating contact as a proper XPBD constraint and recovering the force from its multiplier as λ/h^2, which is timestep-independent by construction at convergence, did not resolve it: with a near-rigid contact the multiplier re-encodes the same positional correction, and sweeping the contact compliance across five orders of magnitude left a 50–80% spread. Peak bending stress $\sigma = E\kappa r$, computed from the deflected centreline so that no solver-reported force enters, converged at a fixed timestep and a single speed but not across speeds, since a fast pass resolves a correspondingly shorter contact event. Holding the spatial step constant and refining every data point individually, all points moved and several by more than 150%.

The diagnosis is that the peak of a stiff impact on a lightly damped discrete elastic chain is not a converged quantity. The manipulator is kinematically driven, so contact is effectively an immovable object striking a compliant one, which excites high-frequency modes of the particle chain. The model's only dissipation is a global velocity damping acting nearly equally on all modes, so as the timestep shrinks, progressively higher modes are resolved and ring, and the peak curvature sharpens rather than settling. This is a modelling gap rather than a solver defect: the solver is being asked for a quantity the model does not define.

The remedy is strain-rate-dependent damping in the constitutive model. Plant tissue is strongly viscoelastic and dissipates preferentially at high strain rate, which is precisely the mechanism absent here. We identify this as the primary avenue for future work. The qualitative claim is unaffected: contact forces and stresses rise monotonically with approach speed, and a deformable model yields contact information a rigid one cannot produce at all.

VIII. Limitations

We state these plainly, in the body text rather than only in a closing section.

No validation against measured plants. Every check in this paper is against analytic beam theory or a second implementation of the same algorithm. Material constants are drawn from published ranges for green herbaceous tissue rather than measured from a specific crop. Matching a named species requires bending tests on real stems, which we have not performed. Accordingly we claim numerical correctness, not biological fidelity.

Aerodynamics are a drag model, not a flow solve. There is no wake and no plant-to-plant sheltering; every plant sees the prescribed field. In a dense canopy, downwind plants would in reality be sheltered by upwind ones.

Leaves are one-dimensional chains. They carry the correct order of drag area but cannot twist, furl, or flutter about their own axis—and reconfiguration of this kind is a significant part of how real foliage sheds aerodynamic load at high wind speed.

The model is purely elastic. There is no plastic deformation and no breakage, so the simulation can report the load that would break a stem but not what follows.

Impact metrics do not converge, as established in Section VII.

IX. Conclusion

We have presented a biomechanical model of slender herbaceous row crops under wind, intended for agricultural robot simulation, in which the plant's bending response follows from a tapered flexural rigidity profile rather than from tuned parameters. Wind is modelled as drag on relative velocity, which we show is necessary for the deflection to reach equilibrium at all. The model is validated against a closed-form cantilever solution and an independent implementation.

Two findings generalise beyond this application. A pinned root is not a clamped root, and the distinction determines whether a simulated plant stands or topples irrespective of its stiffness. And chain topologies are poorly served by colour-parallel constraint solving, which at equal cost produced a stem $4.4\times$ more compliant than specified—a discrepancy that presents as a modelling error and is a convergence error.

Two items of immediate work remain. Validation against real plant deflection under known wind, for which the experimental protocol and analysis pipeline are prepared but no measurements have yet been taken; and strain-rate-dependent damping, without which the impact-severity question posed in Section VII cannot be answered. The latter is the more interesting of the two, since it converts a quantity the model currently cannot define into one a planner could act on.

Reproducibility

Every numerical result in this paper is produced by a script in the released repository, and the parameter table and result values in the manuscript are generated from the code rather than transcribed. Values that have not been measured are rendered as explicit placeholders rather than filled with estimates. The validation, convergence and benchmark experiments in Sections IV–VII are each reproduced by a single command.

Acknowledgment

Portions of this manuscript and of the accompanying software were prepared with the assistance of Claude (Anthropic). The system was used for drafting prose throughout the manuscript, for implementing components of the simulation code and the analysis and plotting scripts, and for producing the figures from simulation output. All numerical results reported here were generated by executing the released code. The authors verified the derivations, the experimental results, and every citation, and are responsible for the content of this paper.

References

[1] J. Deng, S. Marri, J. Klein, W. Pałubicki, S. Pirk, G. Chowdhary, and D. L. Michels, "Gazebo plants: Simulating plant-robot interaction with cosserat rods," *arXiv preprint arXiv:2402.02570*, 2024.

[2] H. Munn, "Orchardbench: A physically-grounded, gpu-parallel apple-orchard simulation benchmark for agricultural robotics," *arXiv preprint arXiv:2607.06337*, 2026.

[3] "Agri-sim: Agricultural simulation platform for embodied intelligence evaluation in greenhouse robotics," *arXiv preprint arXiv:2608.29100*, 2026, author list to be completed from the arXiv listing before submission.

[4] "Cropcraft: A procedural world generator for robotic simulation of agricultural tasks," *arXiv preprint arXiv:2511.02417*, 2025, author list to be completed from the arXiv listing before submission.

[5] M. Macklin, M. Müller, and N. Chentanez, "Xpbd: Position-based simulation of compliant constrained dynamics," in *Proc. 9th Int. Conf. on Motion in Games*, 2016, pp. 49–54.

[6] M. Müller, B. Heidelberger, M. Hennix, and J. Ratcliff, "Position based dynamics," in *Journal of Visual Communication and Image Representation*, vol. 18, no. 2, 2007, pp. 109–118.

[7] M. Macklin, K. Storey, M. Lu, P. Terdiman, N. Schmid, and Others, "Small steps in physics simulation," in *Proc. ACM SIGGRAPH/Eurographics Symposium on Computer Animation*, 2019, verify the full author list before submission.

[8] M. Macklin, "Nvidia warp: A python framework for high-performance gpu simulation," 2022, https://github.com/NVIDIA/warp.